JustCmul8 Viva: Detailed Answers

Each answer is about one minute to speak

1. Requirements

Q1. What are requirements and how did you collect them?

Requirements are the list of things the system must do and how well it must do them. We collected them in the first phase of the project. We studied existing tools like AnyLogic, Arena, Simul8 and the SimPy library and noted their problems: high cost, need for coding and need for installation. We also thought about who will use the system, so we made three personas: a student, a lecturer and a small business owner. From this we wrote a Software Requirements Specification. It has the purpose, user classes, operating environment, design constraints, functional requirements and non functional requirements. Each requirement got an ID, so we could link it later to modules and test cases.

Q2. What are functional requirements? Give examples from your project.

Functional requirements say what the system must do. We have 27 of them, numbered FR 01 to FR 27, and grouped by module. For example, FR 01 says users can register and log in with email or with Google and GitHub. FR 05 says the project saves to the cloud by itself, without a Save button. FR 07 says the canvas supports 15 node types. FR 12 says the graph becomes a Python SimPy script that runs in the browser. FR 14 says the run happens in a Web Worker so the screen does not freeze. FR 16 says the AI turns plain English into a graph. FR 22 says Monte Carlo can run 10, 20, 50 or 100 times with a 95 percent confidence interval. FR 26 says share links are read only and can be revoked.

Q3. What are non functional requirements? Give examples.

Non functional requirements say how well the system must work, not what it does. We have 11 of them, NFR 01 to NFR 11. Performance: moving and connecting nodes must feel instant. Responsiveness: a running simulation must never freeze the screen. Correctness: results for standard queues must match queueing theory within a fixed tolerance. Security: passwords are stored only as salted hashes, all traffic uses TLS and secret keys never go to the browser. Access control: Row Level Security on every table. Usability: a first time user with no technical background can use it. Others are reliability, portability across browsers, maintainability, data integrity with cascade delete, and 99 percent availability.

Q4. How are requirements shown in your report and how are they linked to the rest?

Functional requirements are in Table 2.2 and non functional requirements in Table 2.3. Each row has an ID, the module and the requirement in one sentence. Then in Chapter 4, Table 4.1 shows which module covers which requirements, for example the Simulation Engine covers FR 12 to FR 15. Every module section also ends with a small table of requirements covered, source files and the developer. In Chapter 7 each test case has a requirement column, so TC 22 checks FR 12. Finally Table 7.6 is a traceability table that links each group of requirements to its proof. So you can follow one requirement from the specification, to the module, to the test.

Q5. Who are your user classes and personas?

We have three user classes. The registered user or project owner has full control of their own projects. The collaborator joins a project live and can see presence, cursors and edits. The guest viewer opens a public share link and can only read, with no account needed. None of them need programming skill. We also used

three personas to design the screens. The student wants to see M/M/1 behaviour in a minute using a template. The lecturer shows congestion in class with the animation and gives a share link. The small business owner describes the shop to the AI, runs Monte Carlo and checks the cost panel before spending money. All three follow the same flow: log in, build, run, read results, export or share.

2. Modules

Q6. How many modules are there and how did you divide them?

There are 12 modules and we divided them equally, six each. Khushi Singh made Module 1 Platform and Data Layer, Module 3 Node Configuration Panel, Module 5 AI Assistant, Module 7 KPI Dashboard, Module 9 Digital Twin and Module 11 Executive Report and Share. Mohit Gupta made Module 2 Visual Graph Editor, Module 4 Simulation Engine, Module 6 AI Optimiser, Module 8 Monte Carlo, Module 10 Template Gallery and Module 12 Real Time Collaboration. The owner of a module designed, coded, tested and documented it, and the other member reviewed it. Shared work was requirements, architecture, database design, integration, testing and the report. Our synopsis had 11 modules; during development we merged login, project and profile into one module, and the AI Optimiser, Monte Carlo and Digital Twin became their own modules.

Q7. Explain Module 1, Platform and Data Layer.

This module decides who the user is, what they own and where their work is saved. Everything else depends on it. It handles sign up and log in with email and password or with Google and GitHub, password reset, sign out and account deletion. It lets the user create, rename and delete projects in any of the six domains. The graph saves to the cloud by itself while the user edits. It also has the dashboard that lists projects and a settings page. It is built on Supabase. Next.js middleware refreshes the session and protects pages. The important rule is that security is not in the browser. Every table has a Row Level Security policy, and when an account is deleted, all its data is deleted by cascade.

Q8. Explain Module 2, Visual Graph Editor.

This is the canvas where the user draws the model. The user picks nodes from a palette, drags them on the canvas and joins them with arrows called edges. It supports 15 node types like source, queue, resource, decision and sink, in six domains. Each domain uses its own words, so a bank model shows Teller and a factory model shows Machine, even though both are the same resource node inside. The user can zoom, pan, fit to view and delete. Before a run, the graph is checked, and if a node is not connected, the run is stopped and that node is named. We used React Flow for the canvas and Dagre for automatic layout of generated and template graphs.

Q9. Explain Module 3, Node Configuration Panel.

Placing a node says what it is; configuring it says how it behaves. When the user selects a node, a side panel opens and shows only the fields for that node type. The user can set arrival rate, service time distribution and its values, capacity, queue order, routing probabilities and delays. There are five distributions: exponential, uniform, normal, Poisson and deterministic. There are also cost fields like hourly resource rate, holding cost and waiting penalty, used later in the Cost and ROI panel. All values are checked when typed and again before a run, for example capacity cannot be zero or negative. Settings are stored as JSON, so new node types do not change the database.

Q10. Explain Module 4, Simulation Engine.

This is the technical heart of the project. A run has four stages. First, the graph is checked for structure. Second, the code generator walks through the graph and writes a Python SimPy script, where nodes become resources, stores and processes, and edges become routing. Third, the script is sent to a Web Worker, where Pyodide runs it in the browser using WebAssembly. Fourth, the results and event log come back to the main screen for analysis. No simulation runs on the server. The user sets the run time in seconds, minutes, hours or days, can stop the run and change speed. There is also a code inspector that shows the Python code and can export it as a Jupyter notebook.

Q11. Explain Module 5, AI Assistant.

This module helps a user who does not know how to draw a model. The user just writes in plain English, for example a clinic with two doctors and a waiting room, and the AI returns a first version of the model. The request goes through a Next.js API route to Google Gemini, so the API key stays on the server. The prompt includes the node types and their settings, so the AI only makes graphs our engine can run. Every generated graph is checked for unknown nodes and loose edges. Then it is shown as a preview, and the project changes only when the user accepts. A model router keeps a list of AI models and switches to the next one if one fails. Chat history is saved per project.

Q12. Explain Module 6, AI System Optimiser.

Module 5 helps build a model; this module helps improve it. After a run, it looks at utilisation, waiting time and flow at each node. It finds the node that is really holding the system back and explains it in simple words. A flow analysis step first decides where adding capacity will actually help, because adding capacity to a node that is not the limit does nothing. Only that node is sent to the AI to be explained. It then suggests one clear change, like one more server or a bigger buffer. The user can apply it with one click, as a normal graph edit. The old graph is kept, so the before and after can be compared in Monte Carlo.

Q13. Explain Module 7, KPI Dashboard and Results.

A raw event list is not an answer, so this module turns the run into useful numbers. For each node it shows utilisation, average and percentile waiting times, throughput and queue length. It marks the bottleneck, which is the node with the longest average wait. It checks the run with Little's Law and gives a health score from 0 to 100, made of completion 35, utilisation balance 30, stability 20 and tail waiting time 15. There are tabs like Executive Summary, Flow and Balance Check, Traffic Heatmap, Line Buildup Timeline, Station Counters, Person or Item Journey, Live Activity Log, and Cost and ROI. The calculations are plain TypeScript, so they can be tested alone.

Q14. Explain Module 8, Monte Carlo and Scenario Comparison.

One random run is only one sample, not a reliable answer. So this module runs the same model 10, 20, 50 or 100 times, each with different random numbers. For each metric it calculates the mean, standard deviation and the 95 percent confidence interval using mean plus or minus 1.96 times s divided by root n. The results are drawn as bands, so you can see the range and not just one number. It also compares a baseline with a changed scenario, for example before and after an AI fix, and shows the difference in each metric. Progress is shown while it runs. This helps the user see if an improvement is real or just random noise.

Q15. Explain Module 9, Digital Twin Playback.

Numbers tell you there is congestion; animation shows it. After a run, this module replays it as a 2D animation where entities move through the nodes and edges. Queue build up is highlighted, so you can see lines grow. There are video style controls: play, pause, step and speed up to 10 times, and you can jump to any point. It does not run the simulation again; it replays the saved journeys of each entity. So jumping and speed changes are instant, and the animation always matches the numbers on the dashboard. It is drawn with Pixi.js on WebGL, because moving hundreds of objects with normal HTML elements would be too slow.

Q16. Explain Modules 10, 11 and 12 together.

Module 10 is the Template Gallery. It gives ready starter models for all six domains, so a new user does not start with a blank screen. A template is copied into the project, so editing it never changes the original. Module 11 is Executive Report and Share. It exports a PDF report, the graph as JSON, results as CSV and the Python code as a Jupyter notebook. It also makes read only share links that can be revoked; viewers never touch a table, only two small database functions. Module 12 is Real Time Collaboration. Many users can work on one project and see each other's avatars, cursors and edits, using Supabase Realtime channels that are protected by the database.

Q17. What are the main functionalities of the whole system?

A user signs up and creates a project in one of six domains. They build a model by dragging nodes or by asking the AI in plain English, or they start from a template. They set values for each node in the side panel. They press Run, and the model becomes a Python SimPy program that runs in the browser without freezing the screen. The dashboard then shows utilisation, waiting times, throughput, the bottleneck, a Little's Law check, a health score and cost. The AI optimiser suggests a fix. Monte Carlo runs the model many times to give a confidence range. The digital twin replays it as an animation. Finally the user can export a PDF, JSON, CSV or notebook, share a read only link, or work live with teammates.

3. Planning

Q18. Explain your Gantt chart.

A Gantt chart shows every task as a bar on a calendar, so you can see when it starts, how long it takes and which tasks overlap. Our chart runs week by week from July 2026 to March 2027. Each phase has a colour: blue for analysis, purple for design, red for development, orange for testing, grey for documentation and green for review and deployment. Red diamonds mark milestones. July has initiation, requirements, literature survey and system analysis. Design follows in late July. Development fills August to October. The simulation engine starts early, in the second week of July, because it was the biggest risk. Testing overlaps development. The original plan ended with deployment in December. We then added January to March 2027 for regression testing, bug fixing, Parallel Monte Carlo, seeded runs and a final v1.1 release. Documentation runs through the whole period.

Q19. Explain your Activity Network Diagram and critical path.

The Gantt chart shows when tasks happen; the activity network shows in what order and which tasks can run together. It is drawn in UML with activity nodes, arrows, decision diamonds, and fork and join bars. It starts with initiation, requirements, literature survey and analysis. A decision checks if requirements and the database schema are fixed; if not, we go back. Then development forks into three parallel lanes:

backend and API, simulation, and frontend. A join bar brings them together for integration. After testing, another decision checks for defects and loops back to bug fixing until the build is clean. Then UAT, review and deployment. The critical path is the longest chain of tasks: requirements, analysis, database design, backend, integration, testing, UAT and deployment. If any of these is late, the project is late.

Q20. Which development model did you use and why?

We used the iterative and incremental model, a mix of waterfall and agile. Our requirements were mostly stable, so pure Agile was not needed, and we also needed proper documents for college. But waterfall was risky, because we did not know if SimPy could run in the browser, and in waterfall we would find this out very late. Spiral was too heavy for two people. So we did analysis and design first like waterfall, then built an early prototype of the engine to remove the biggest risk, which is milestone M4. After that we built one module at a time, and each module went through design, code, test and integration. Testing and documentation ran the whole time. We tracked the work in Jira with two week sprints.

4. Diagrams

Q21. Explain the Context Diagram.

The context diagram is DFD level 0. It shows the whole JustCmul8 system as one single process in the centre, and the outside entities around it with the data that flows in and out. Nothing inside the system is broken down; its job is only to show the boundary. There are four external entities. The Simulation Builder is the registered user who sends login, project, graph, run, AI and share requests and gets back results, dashboards and files. The Shared Link Viewer is an anonymous visitor who can only view, so nothing they do changes stored data. Supabase Auth handles login and sessions. The AI and Simulation Engine Services stand for Google Gemini and the Pyodide SimPy runtime. The database is inside the boundary because our app controls it.

Q22. Explain the DFD Level 1.

Level 1 opens up the single process of the context diagram into seven smaller processes and shows the data stores they read and write. P1 Account and Access handles sign up, login, session and account deletion with Supabase Auth. P2 Project Management creates, renames and deletes projects and loads templates, using the Projects store. P3 Graph Design adds and edits nodes and edges and saves them to the same store. P4 Simulation Execution reads the graph, runs the engine and writes to the Simulation Runs store. P5 AI Assistance talks to Gemini and saves to the Chat History store. P6 Sharing and Export makes share links and exports, using the Project Shares store. P7 View and Results shows results and the read only view. It is the only process the anonymous viewer can reach.

Q23. Explain the ER Diagram.

The ER diagram shows how data is stored and how the tables are related. We used Chen notation: rectangles are entities, ellipses are attributes and diamonds are relationships, with cardinality marked. The User is in the centre. User has a one to one relationship with Profile, which keeps the display name, avatar and default domain; we kept it separate from login data. One User owns many Projects; each project has an id, owner, name, domain, the graph as JSON and update time. A User also creates many SimulationRuns, ChatHistory messages and ProjectShares. One Project has many runs, chats and share records. A run saves its bottleneck, so old runs keep their result. A revoked share is not deleted, it just gets a revoked time.

Q24. Explain the Class Diagram.

The class diagram shows the object model as written in TypeScript: class names, typed attributes, methods and links. It has three rows. The top row has account and project classes: User, Profile, Project and ProjectShare, with links like User has a Profile, User owns Projects and Project is shared through ProjectShare. The middle row has graph and execution classes. SimNode has id, type, label, a JSON parameter object and position. SimEdge joins two nodes. SimulationRun has status, speed and results. ChatHistory saves AI chats. The bottom row has measurement classes: NodeStats, SimLog, EntityJourney and EntityJourneyStep. Journeys are used by the digital twin to replay a run without running it again.

Q25. Explain the State Transition Diagram.

This diagram shows the states of one node while a simulation runs. A node is in exactly one state at a time, and the event scheduler moves it between states. It starts in IDLE with nothing to do. When an entity arrives it goes to QUEUED, where the entity waits for space. When the resource is free it goes to PROCESSING and holds the entity for a random service time. Then it goes to COMPLETED and sends the entity on. If no one else is waiting, it returns to IDLE. There are two special exits from PROCESSING. BLOCKED happens when the next node is full, which shows how congestion moves backwards. FAILED happens on an interrupt, a higher priority job, or an error. Each state maps directly to a SimPy action.

Q26. Explain the Use Case Diagram.

The use case diagram shows the actors and what each one can do, from the outside view. There are two human actors. The Simulation Builder is a registered user who can sign up, log in, manage the account, create, rename and delete projects, design the graph, set node values, load templates, start and stop a run, change speed, view the dashboard, chat with the AI, export a report and create or revoke a share link. The Shared Link Viewer is anonymous and can only open a shared link, view the graph, view results and the entity journey, and see a notice if the link is revoked. Three external systems are connected with include: Supabase Auth for login, the SimPy engine for running, and Gemini for the AI. We also wrote four detailed use case specifications.

Q27. Explain the Activity Diagram.

The activity diagram shows the full path a user takes in the running system, from login to export. It uses swimlanes, which are columns for each actor: the Simulation Builder, the system, Supabase Auth, the Pyodide SimPy engine and the Gemini API, so it is clear who does each step. The user fills the login form and Supabase checks it; a decision sends them back if it is wrong. Then the dashboard loads, the user creates a project and designs the graph. A second decision checks if the graph is connected and sends them back if not. Then the system makes the SimPy script, the engine runs it and the dashboard shows results. A third decision asks if AI help is wanted. Finally the user exports or shares, and the flow ends.

Q28. Explain the Component, Collaboration and Deployment Diagrams.

The component diagram shows the software parts and how they depend on each other. In the browser we have the graph editor, configuration panel, dashboard and digital twin; the Zustand state store; and services like the code generator, analytics and export. The simulation worker is separate because it runs in a Web Worker. The collaboration diagram shows one simulation run as ten numbered messages: start run, build script, send to worker, run Python, return results, analyse, show KPIs, save to store, save to Supabase and show the bottleneck. The deployment diagram shows the physical setup: the user's browser with Pyodide, the Vercel server for Next.js, and Supabase with PostgreSQL, Auth and Realtime, connected through HTTPS.

Q29. Explain your Architecture Diagram.

Our system is built in layers, and each layer only uses the layer below it. The presentation layer is Next.js, React and Tailwind: the pages, dashboard and workspace. It has no business logic. The application and state layer is Zustand with separate stores for the graph, simulation, interface and presence. The domain and service layer has plain TypeScript modules: code generator, analytics, Monte Carlo, report and export. The execution layer is a Web Worker with Pyodide and SimPy, so the screen never freezes. The bottom layer is Supabase Auth, PostgreSQL with Row Level Security, Realtime and the Gemini API. The unusual part is that simulation runs on the user's computer, not the server, which keeps cost low.

Q30. Explain your database tables.

We use PostgreSQL through Supabase. Users are kept by Supabase in auth.users. We have five app tables. profiles keeps display name, avatar and default domain. projects keeps the owner, name, domain and the whole graph as one JSONB column. simulation_runs keeps the run time, entities arrived and completed, the bottleneck and the results. chat_history keeps AI messages. project_shares keeps the share token and the revoked time. Every primary key is a UUID. Foreign keys use ON DELETE CASCADE, so deleting an account removes everything. Every table has Row Level Security so a user only sees their own rows. Anonymous viewers can only call two functions: one returns a shared project, the other checks if a link is active, revoked or invalid.

5. Testing

Q31. How did you test your project? Explain the strategy.

We tested in two parts. A simulator can be easy to use but still give wrong numbers, so first we checked the engine against mathematics, and second we tested the application features. Testing ran alongside development, not after it. The strategy is layered: small, fast tests at the bottom and bigger, slower ones at the top. Unit tests check samplers and analytics on their own. Structural tests check that bad graphs are refused. Analytical verification compares results with M/M/1, M/M/c and Little's Law. Invariant tests check laws on a complex hospital model. Integration tests check the full path from store to dashboard. Security tests hit the database directly. Cross browser and user acceptance tests come last.

Q32. Explain engine verification testing and its results.

We ran the generated SimPy script with five independent replications for each benchmark, over 36,000 simulated seconds, and compared the answers with queueing theory. B1 is an M/M/1 queue at 80 percent utilisation. B2 is an M/M/3 queue. B3 is an overloaded queue. B4 checks time units. B5 checks Little's Law. Each check had a tolerance fixed in advance. All 28 out of 28 checks passed, and no result was more than 2.1 percent away from theory. For example in M/M/1 the theory utilisation is 0.8000 and we got 0.7979; theory wait is 4.000 seconds and we got 4.005. In the overloaded test the health score fell to 45.8, so the system correctly reports a failing system.

Q33. What is invariant testing and what was the hospital model?

Some models are too complex for a formula, so we test laws that must always be true. These are called invariants. Our hospital model has patients arriving at 1.2 per second, a triage nurse, and a decision node that sends 70 percent to minor treatment and 30 percent to major treatment. We ran it with and without patients leaving the queue. We checked that nothing is created or lost, that entities in equal entities out

plus those still waiting at every node, that the 70 and 30 split is followed, that completed equals the total of both sinks, and that utilisation is close to expected. All 22 out of 22 invariant checks passed.

Q34. Explain your functional test cases with examples.

We wrote 50 functional test cases, TC 01 to TC 50, covering every module. Each one has the scenario, the input, the expected output and the requirement it checks. For example TC 01 registers with a valid email and expects the account and profile to be created. TC 12 tries to read another user's project and expects no rows because of Row Level Security. TC 17 runs an empty graph and expects the run to be refused with a message. TC 27 asks the AI for a clinic with two doctors and expects a valid graph. TC 33 builds a line with one slow stage and expects it to be flagged as the bottleneck. TC 45 opens a revoked link and expects a clear notice. The status column is filled when each case is run.

Q35. Explain negative, security and acceptance testing.

In negative and boundary testing we gave wrong inputs on purpose: zero or negative capacity, zero duration, empty graphs, unconnected nodes and broken AI replies. The app must refuse them with a clear message and not crash. In security testing we skipped the screen and sent requests directly to the database. We tried to read another user's project, write using only a share token, and join a collaboration channel without access. All were blocked by the database. In cross browser testing we ran one model on Chrome, Edge, Firefox and Safari and checked phone width. In user acceptance testing, users like our personas built a model with no help. Bugs were logged in Jira, fixed, retested and linked to the fix.

6. Technology

Q36. Explain your technology stack in short.

The frontend is Next.js 16 with React 19 and TypeScript, styled with Tailwind CSS. The drag and drop canvas uses React Flow, and Dagre arranges graphs automatically. The simulation uses SimPy, a Python library, which runs in the browser through Pyodide and WebAssembly inside a Web Worker. App state is kept in Zustand. Charts use Recharts and the animation uses Pixi.js with WebGL. PDF reports are made with jsPDF and html to image. The backend is Supabase, which gives PostgreSQL with Row Level Security, login and Realtime channels. The AI is Google Gemini through our own model router. The app is hosted on Vercel, code is on GitHub, and tasks were tracked in Jira.

7. Users and Competitors

Q37. Who are your target users and how do they benefit?

Our users are people who want to test a decision before spending money, but cannot afford costly tools or do not know coding. First, students: they can see queueing theory working in a minute using a template, instead of only solving formulas. Second, lecturers: they can show congestion live in class with the animation and give students a share link. Third, researchers: they get a real SimPy engine and can export the Python code as a notebook to check results. Fourth, small business owners and startups: for example a shop owner can describe the shop to the AI, run Monte Carlo and check the cost panel before adding a counter. All of them need only a browser, no licence and no installation.

Q38. Who are your competitors?

Our competitors fall into three groups. The first group is commercial simulation software like AnyLogic, Arena and Simul8. They are powerful, but they cost thousands of dollars per user per year and must be

installed. The second group is free programming libraries, mainly SimPy in Python. It is free and accurate, but the user must install Python and write and debug code before getting any result. The third group is drawing tools and spreadsheets like Visio, Lucidchart and Excel. They are easy, but a diagram does not run and a spreadsheet cannot model a queue that changes over time. JustCmul8 sits in the gap between them: free like SimPy, easy like Visio, and a real simulator like AnyLogic.

Q39. How is your project different from AnyLogic, Arena or Simul8?

The first difference is cost. Those tools charge thousands of dollars per seat every year, while JustCmul8 is free and uses an open source stack. The second difference is installation. They are desktop software, so they are hard to use in college labs or on office laptops where you cannot install anything. Ours runs in any modern browser. The third difference is ease. They usually need formal training, while we use simple domain words like Teller or Machine, a template gallery and an AI assistant that builds a model from plain English. We also show the generated Python code, which they do not. To be fair, they are more powerful for very large industrial models, and that is in our limitations and future scope.

Q40. How is your project different from SimPy?

We actually use SimPy inside our project, so the engine is the same and the results are just as accurate. The difference is how you use it. With plain SimPy you must install Python, set up an environment, learn generator functions and resource requests, write the model as code and fix errors yourself. With JustCmul8 you drag nodes, fill a form and press Run, and our code generator writes the SimPy program for you. On top of the engine we add things SimPy does not give: a results dashboard, bottleneck detection, Little's Law check, health score, Monte Carlo with confidence bands, cost analysis, animation, AI help, sharing and live collaboration. And the code is still visible in the inspector.

Q41. Why not simply use Excel, Visio or Lucidchart?

Visio and Lucidchart can draw a process very nicely, but the drawing is only a picture. It cannot tell you how long the line will be at 3 pm or what happens if you add one more counter. Excel can calculate averages, but real systems are random. Customers come in bunches and service times change. Averages hide this, so a counter that looks fine on average can still have a long queue. Excel also has no natural way to model an event queue, a resource that is busy, or entities waiting for each other over time. A discrete event simulation handles all of this. JustCmul8 gives the ease of a drawing tool but actually runs the diagram as a simulation.

Q42. Where can your project be used in real life?

It supports six domains. In human queues, like banks, hospitals, clinics and ticket counters, you can test how many staff are needed. In vehicle traffic, like toll plazas, signals and fuel pumps, you can study waiting time and flow. In liquid and material flow, like tanks and pipelines, you can study levels and flow rates. In manufacturing, you can find the slow machine on an assembly line. In logistics and warehousing, you can study picking, packing and dispatch. In networks, you can study packets passing through routers and links. In teaching, it helps students understand queueing theory. The common idea is the same: test a change on the computer before doing it in real life.

Q43. What is the unique point or novelty of your project?

The main novelty is that a real Python simulation engine runs fully inside the browser. Most web tools either fake the simulation with simple JavaScript animation, or run it on a costly server. We turn the diagram into a real SimPy program and run it on the user's own computer using Pyodide and

WebAssembly in a Web Worker. So it is free to scale, needs no installation and the screen does not freeze. Second, we proved the engine is correct, with 28 out of 28 checks against queueing theory. Third, we combined many things in one place: no code modelling, AI model building, AI fix suggestions, Monte Carlo confidence intervals, animation, cost analysis, sharing and live collaboration.

8. Other Common Questions

Q44. Why did you choose this project?

We chose it for three reasons. First, simulation is a very useful way to make decisions, but the people who need it most, like students, small businesses and colleges, cannot afford commercial tools. Second, the best free option, SimPy, needs coding, so most people cannot use it. Third, technology has now changed. WebAssembly lets real Python run in a browser at good speed, so for the first time we could run a real simulation engine on the user's device with no server and no installation. We also wanted to use modern ideas like AI help and live teamwork. This project also used many subjects we studied: data structures, databases, software engineering, probability and statistics.

Q45. What were the biggest challenges and how did you solve them?

The biggest challenge was running Python SimPy in the browser. We did not know if it would work, and many modules depended on it. We solved it with Pyodide and built a prototype first, which was milestone M4. Second, a long simulation could freeze the screen, so we moved it to a Web Worker. Third, we had to prove the numbers are correct, so we built a separate test harness and compared results with M/M/1, M/M/3 and Little's Law. Fourth, the AI sometimes gives wrong graphs, so we validate every graph and always ask the user before applying it. Fifth, security for sharing, which we solved with Row Level Security and small database functions.

Q46. Why does the simulation run in the browser and not on a server?

There are three reasons. First, cost: if simulations ran on our server, every extra user and every long run would increase our server bill. When it runs in the browser, the user's own computer does the work, so a thousand users running simulations cost us about the same as a thousand users opening a normal page. Second, no installation: the user only needs a browser, and Pyodide downloads once and is saved. Third, speed and privacy: there is no waiting for a server, and the model does not need to leave the device to be run. The server is only used for login, saving, collaboration and keeping the AI key secret. The limit is that model size depends on browser memory.

Q47. How do you know your simulation results are correct?

We did not just trust the engine, we tested it against mathematics. For simple queues like M/M/1 and M/M/3, queueing theory gives exact answers for utilisation, waiting time and queue length. We ran the same queues in our engine, with five replications each over 36,000 simulated seconds, and compared. All 28 checks passed and no result was more than 2.1 percent away from theory. For a complex hospital model where no formula exists, we checked laws that must always hold, like nothing is lost, and all 22 passed. Every run in the app is also checked with Little's Law. And because the Python code can be exported, anyone can run it again and check.

Q48. How is AI used in your project? Can it make mistakes?

AI is used in two places. The AI Assistant turns a plain English description into a model, and the AI Optimiser explains the bottleneck after a run and suggests a fix. We use Google Gemini through our own

model router, which switches to another model if one fails. Yes, AI can make mistakes, so we never trust it blindly. The prompt includes only our allowed node types. Every generated graph is checked for unknown nodes and loose edges, and bad ones are rejected. The optimiser does not guess; it uses the real computed numbers and a flow analysis step first. And nothing changes without the user's approval. The AI key stays on the server, not in the browser.

Q49. How is user data kept secure?

Security is enforced in the database, not only on the screen. Every table has Row Level Security, which means the database itself checks that the logged in user owns the row. Even if someone changes the frontend code or sends a request by hand, they get nothing. Passwords are handled by Supabase Auth and stored only as salted hashes, and all traffic uses HTTPS. The Gemini API key is kept on the server only. Share links use a long random token, are read only, and can be revoked. Anonymous viewers can only call two small database functions and never touch a table. Collaboration channels are also checked by the database. Deleting an account deletes all its data through cascade delete.

Q50. What are the limitations of your project?

First, model size is limited by the memory of the user's browser, since the simulation runs on their device. Second, Monte Carlo replications run one after another, so 100 runs take longer. Third, the first run needs to download the Pyodide runtime, which takes a little time, though it is saved after that. Fourth, saving, AI and collaboration need internet. Fifth, AI results still need a human to check them. Sixth, we support discrete event simulation only, not full agent based or continuous simulation. Seventh, there is no native mobile app, and placing nodes on a small touch screen is hard. Each of these is planned in our future scope.

Q51. What is the future scope?

In the near term we want Parallel Monte Carlo, which runs replications on many Web Workers at once so 100 runs finish much faster. We also want reproducible runs with a fixed random seed, pause and resume for long runs, roles like owner, editor and viewer, better conflict handling when two people edit the same node, friendlier input checks, and faster first loading. In the medium term we plan custom node plugins, agent based simulation, automatic optimisation that searches for the cheapest setup, an offline classroom mode, a mobile app and a template marketplace. In the long term we want cloud runs for very large models, linking to real IoT data for a live digital twin, and features for whole institutions.

Q52. Explain your cost estimation in short.

We used two methods. COCOMO estimates effort from code size. Our code is 22.19 thousand lines. Basic COCOMO in organic mode gave 62.19 person months, about 12 months and 5 people, which is about 24.88 lakh rupees. But that assumes every line is typed by hand. We used AI coding agents and ready libraries like SimPy, React Flow and Supabase, so we adjusted it to 25 percent: 15.55 person months, about 7 months and 2 to 3 people, which is close to our real work. Function Point Analysis measures features instead: we got 281 function points and about 21.6 person months. A company would spend around 6.7 to 11.7 lakh rupees, while we spent under 20 thousand rupees.

Q53. Explain Monte Carlo and Little's Law in simple words.

A simulation uses random numbers, so one run is like one lucky or unlucky day. Monte Carlo means running the same model many times, like 10, 20, 50 or 100 times, each with different random numbers. Then we take the average and a 95 percent confidence interval, which is the range where the true answer

most likely lies. It is mean plus or minus 1.96 times the standard deviation divided by root n. More runs give a smaller range. Little's Law is L equals lambda times W: the average number of items in a system equals the arrival rate times the average time each item spends inside. We use it after every run to check that the results are consistent.

Q54. What did you learn from this project?

We learned to plan around risk: we tested the hardest part, Python in the browser, first, and that saved a lot of rework. We learned that a simulator must be proven, not just look good, so we tested it against real mathematics. We learned to keep security in the database and not trust the frontend. We learned to divide a big system into layers and modules so each part can be built and tested on its own. We also practised teamwork with Git, code reviews and Jira sprints. And we applied many subjects together: data structures for the event queue and graph, DBMS for the schema, software engineering for planning, and statistics for Monte Carlo.

Q55. Can you walk us through a demo of the project?

First I sign up or log in, and the dashboard shows my projects. I create a new project and choose a domain, for example Human Queues for a bank. I can drag nodes like Customer Arrival, Waiting Line, Teller and Exit, connect them, and set values like arrival rate and service time in the side panel. Or I can open a template, or ask the AI to build it from a sentence. Then I set the run time and press Run. The dashboard shows utilisation, waiting time, the bottleneck and the health score. I can ask the AI optimiser for a fix, apply it, and compare with Monte Carlo. Then I can watch the digital twin, export a PDF and share a read only link.